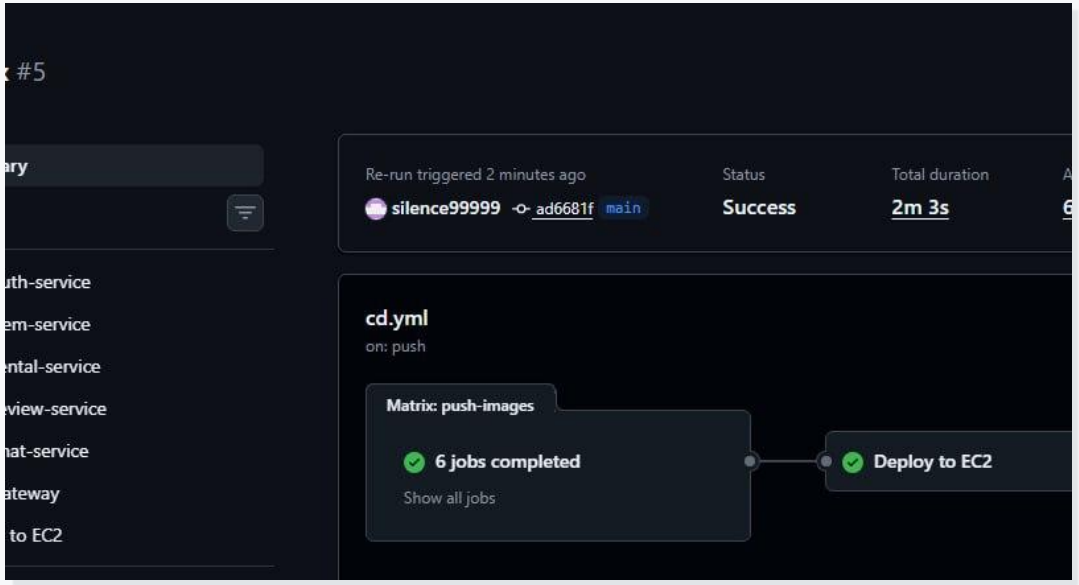
Borrowly

SRE · Microservices · CI/CD

Comprehensive Site Reliability Engineering implementation for a distributed rental marketplace.

Docker · Kubernetes · Terraform · Ansible · GitHub Actions · Prometheus · Grafana



Team
SE-2431 / SE-2430

Project
Peer-to-peer rental marketplace

ASTANA IT UNIVERSITY

Executive snapshot

What the final system demonstrates

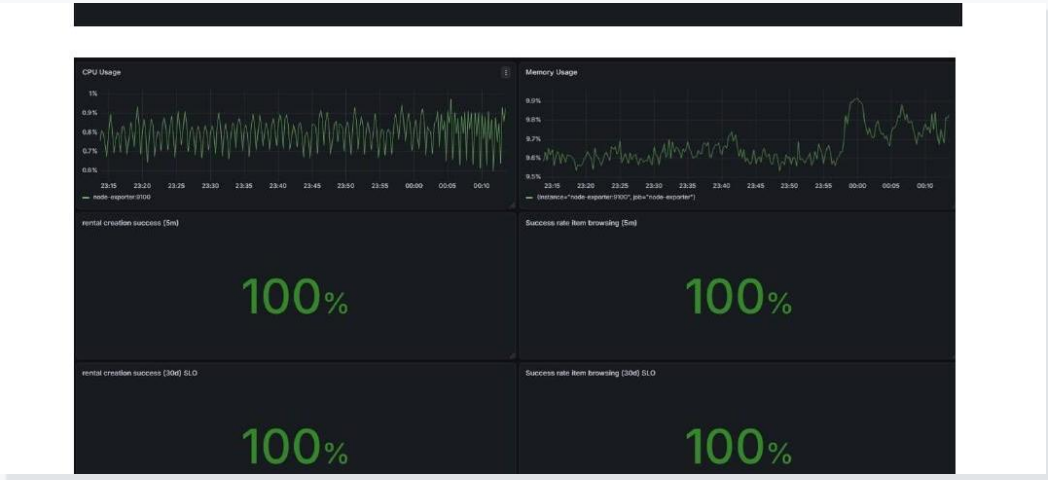
6 services
Auth, item, rental, review, chat, notification

19 green checks
Tests, builds, image pushes and EC2 deploy

2m 03s deploy
End-to-end CD workflow run

12 alert rules
Availability, latency, error budget and host metrics

6 min recovery
Simulated production incident resolved



Live system and pipeline evidence: monitoring dashboard + all checks passing.

ASTANA IT UNIVERSITY

Project goal

Turn a microservices app into a reliable production platform

Reliability by design
Each service exposes health and metrics endpoints, with database isolation and restart policies.

Repeatable operations
Infrastructure, server setup and deployment are automated instead of manual.

Fast feedback loop
GitHub Actions validates each change before deployment to EC2.

The presentation follows the same SRE lifecycle used in the implementation.

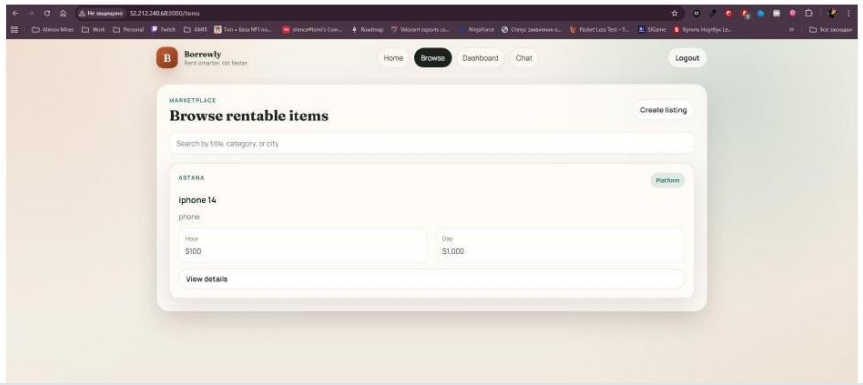
ASTANA IT UNIVERSITY

Borrowly product context

Peer-to-peer marketplace as the reliability target

```
    - db-auth
    - db-items
    - db-rentals
    - db-reviews
    - db-chat
restart: unless-stopped
```

3.3 All 6 Services Running



List items
Users publish rentable items with images and pickup points.

Rent & pay
Rental service manages orders, pricing and statuses.

Chat
User-to-user messaging supports coordination.

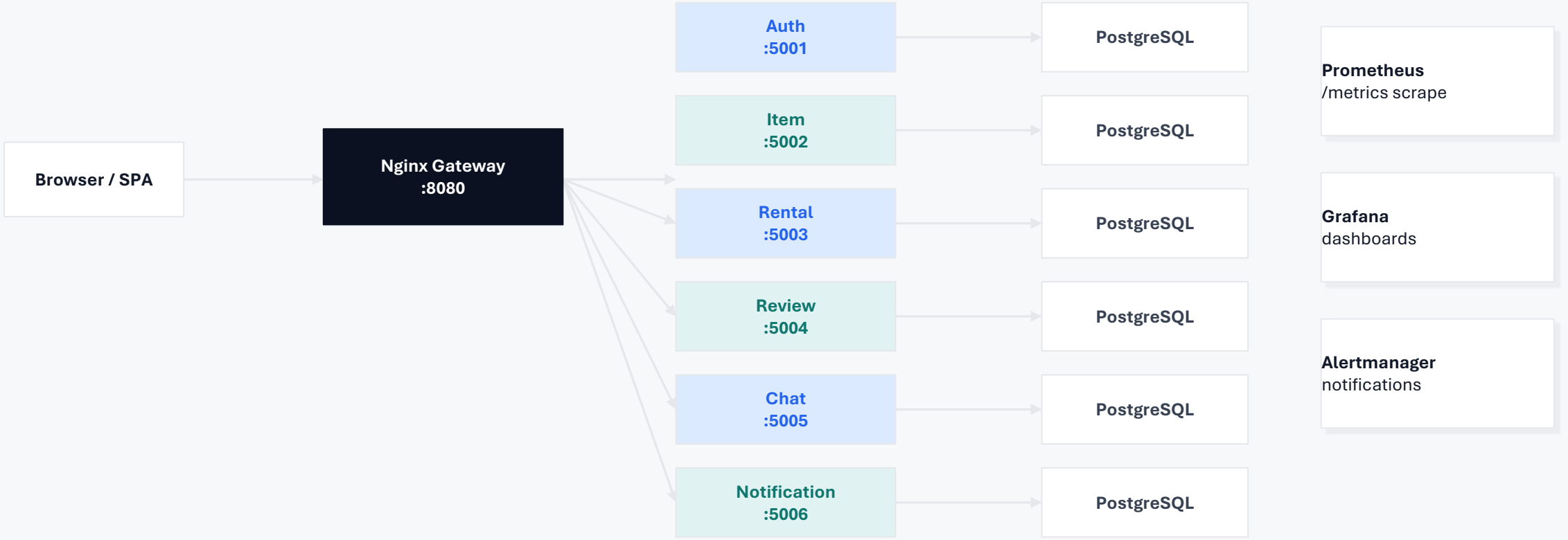
Review
Ratings and reviews close the marketplace loop.

Reliability matters because every feature depends on several services working together through the gateway and databases.

ASTANA IT UNIVERSITY

Architecture

Gateway-first microservices with isolated persistence



The design avoids shared databases between services, making failures easier to isolate and recover.

ASTANA IT UNIVERSITY

Microservices map

Six independent responsibilities, one unified product

5001

Auth
JWT, registration, email verification

5002

Item
CRUD, image upload, pickup points

5003

Rental
Orders, pricing, reminder worker

5004

Review
Item reviews and star ratings

5005

Chat
User-to-user in-app messaging

5006

Notification
Rental confirmations and reminders

Every service is containerized and paired with its own PostgreSQL database, then exposed through the gateway.

Infrastructure & orchestration

The same application can run locally, clustered, and in cloud

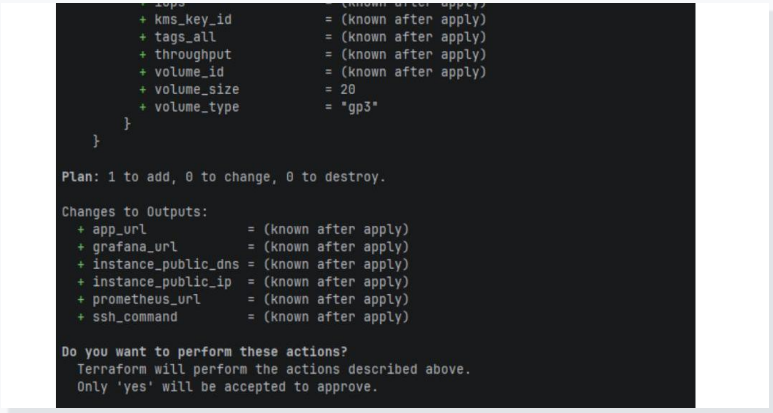
Docker Compose
Local development and full-stack testing

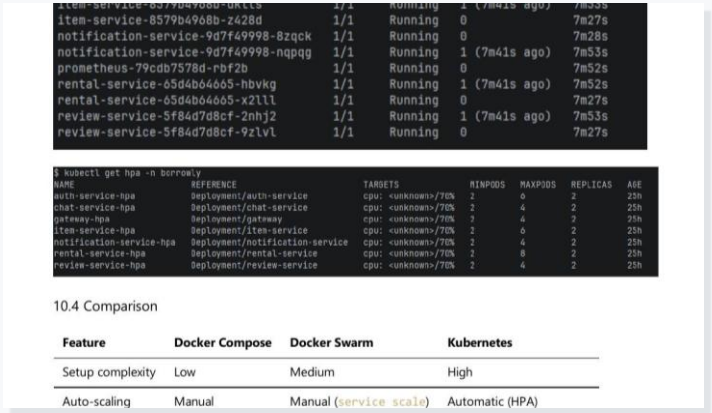
Docker Swarm
Cluster-ready deployment model

Kubernetes + HPA
Production-grade scaling path

Terraform
AWS EC2, security group and outputs

Ansible
Server setup, roles and automated deployment





One-command mindset
Provision, configure and deploy with reproducible scripts.

Secure exposure
Public app gateway; admin tools restricted to admin CIDR.

ASTANA IT UNIVERSITY

Continuous Integration

Parallel checks make every push safe

```yaml
jobs:
  test-go:
    name: Test ${{ matrix.service }}
    runs-on: ubuntu-latest
    strategy:
      fail-fast: false
      matrix:
        service:
          - auth-service
          - item-service
          - rental-service
          - review-service
          - chat-service
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-go@v5
        with:
          go-version: "1.24"
          cache-dependency-path: services/${{ matrix.service }}/go.sum

      - name: vet
        working-directory: services/${{ matrix.service }}
        run: go vet ./...

      - name: test
        working-directory: services/${{ matrix.service }}
        run: go test ./...

  build-frontend:
    name: Build frontend
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-node@v4
        with:
          node-version: "20"
          cache: npm
```

test-go
Matrix over 5 Go services: go vet + go test

build-frontend
Node 20, npm ci, TypeScript check, production build

docker-build
BuildKit builds 6 images with GitHub Actions cache, push disabled

```yaml
    runs-on: ubuntu-latest
    needs: [test-go, build-frontend]
    strategy:
      fail-fast: false
      matrix:
```

Continuous Deployment

Images are pushed, then EC2 is updated over SSH

```yaml
  push:
    branches: [main]
  workflow_dispatch:

permissions:
  contents: read
  packages: write

jobs:
  push-images:
    name: Push ${{ matrix.service }}
    runs-on: ubuntu-latest
    strategy:
      fail-fast: false
      matrix:
        include:
          - service: auth-service
            context: services/auth-service
          - service: item-service
            context: services/item-service
          - service: rental-service
            context: services/rental-service
          - service: review-service
            context: services/review-service
          - service: chat-service
            context: services/chat-service
          - service: gateway
            context: .
            dockerfile: gateway/Dockerfile
    steps:
      - uses: actions/checkout@v4

      - uses: docker/setup-buildx-action@v4

      - uses: docker/login-action@v4
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GHCR_PUSH_TOKEN }}
```

```yaml
      - id: meta
        uses: docker/metadata-action@v6
        with:
          images: ghcr.io/${{ github.repository_owner }}/rent-items-${{ matrix.service }}
          tags: |
            type=sha,prefix=
            type=raw,value=latest

      - uses: docker/build-push-action@v6
        with:
          context: ${{ matrix.context }}
          file: ${{ matrix.dockerfile || format('{0}/Dockerfile', matrix.context) }}
          push: true
          tags: ${{ steps.meta.outputs.tags }}
          labels: ${{ steps.meta.outputs.labels }}
          cache-from: type=gha
          cache-to: type=gha,mode=max

  deploy:
    name: Deploy to EC2
    runs-on: ubuntu-latest
    needs: push-images
    environment: production
    steps:
      - uses: actions/checkout@v4

      - name: Deploy via SSH
        uses: appleboy/ssh-action@v1.0.3
        with:
          host: ${{ secrets.EC2_HOST }}
          username: ${{ secrets.EC2_USER }}
          key: ${{ secrets.EC2_SSH_PRIVATE_KEY }}
```

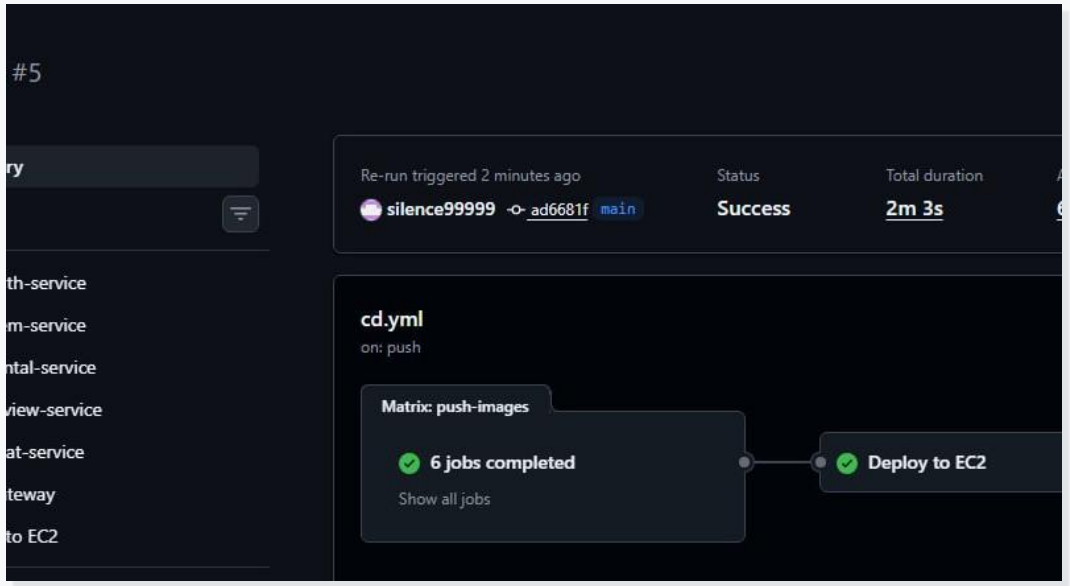
GHCR registry
Images tagged by Git SHA and latest.

Deploy job
SSH to EC2, pull images, restart stack.

Secrets safe
Tokens and keys stored only in GitHub Secrets.

Pipeline evidence

Final run: all checks passed



19
successful checks

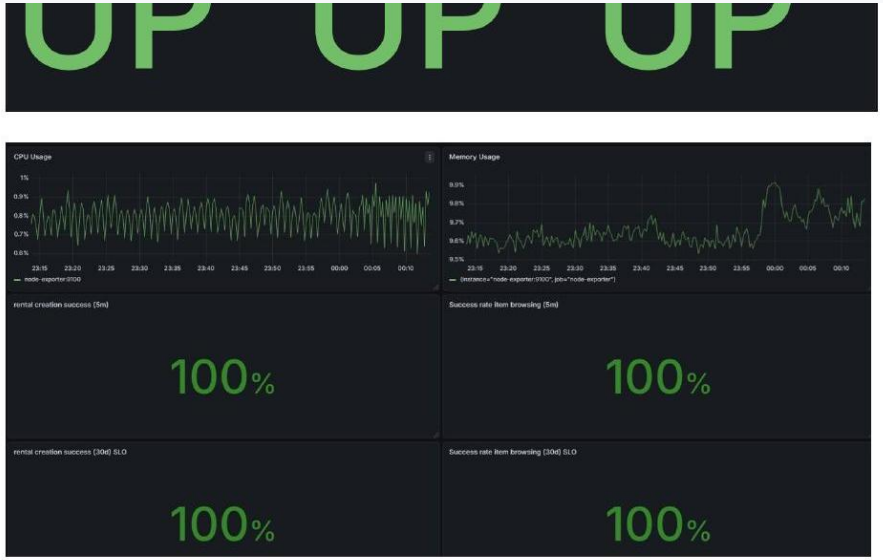
2m 03s
total duration

31s
deploy to EC2

ASTANA IT UNIVERSITY

Observability stack

Metrics, dashboards and alerts close the feedback loop



Prometheus
Scrapes all services and node exporter every 15 seconds.

Grafana
Service health, request rate, latency and resource dashboards.

Alertmanager
Routes incidents to the responsible engineer.

Node Exporter
Host CPU, memory and disk metrics.

Monitoring is not a separate add-on: metrics and health checks are part of every service contract.

ASTANA IT UNIVERSITY

SLIs & SLOs

Reliability targets are explicit and measurable

Objective	Target	Alert threshold
Availability	≥ 99%	up == 0 for 1 minute
P95 latency	≤ 200 ms	Rental P95 > 1s for 5 minutes
Error rate	≤ 1%	5xx rate > 5% for 2 minutes
Rental success	≥ 90%	failure rate > 10% for 2 minutes

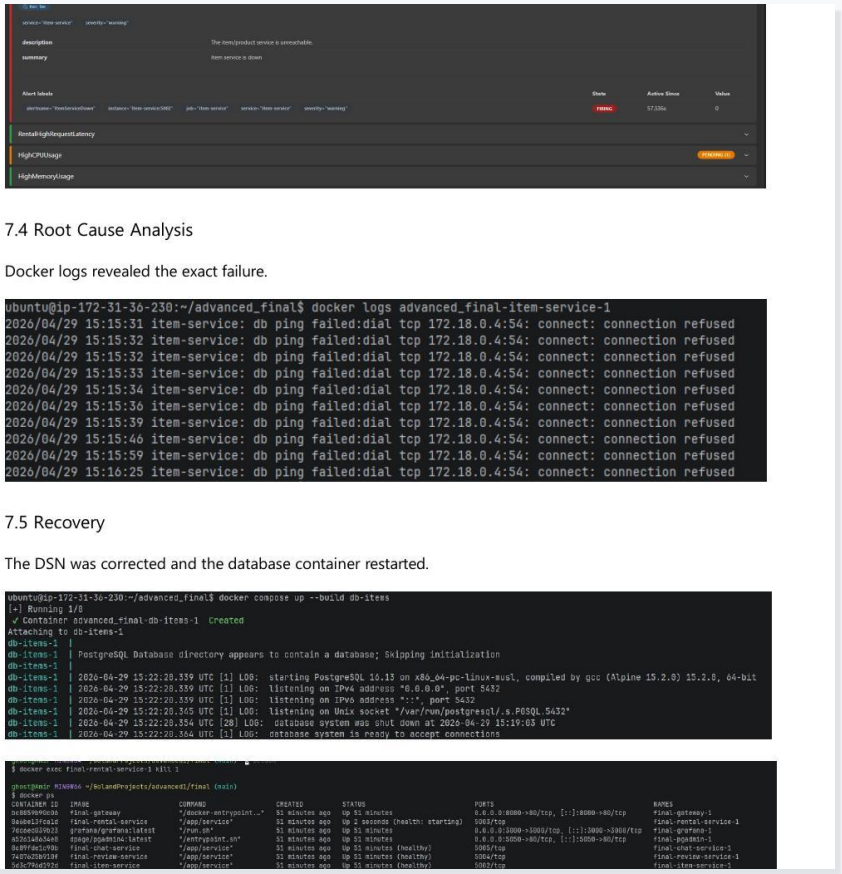
Availability
Prometheus target reachability

Latency
95th-percentile response time

Error budget
5xx errors per second

Incident response

A controlled failure tested detection and recovery



T+0 DB DSN misconfigured

T+1 Prometheus fires alert

T+2 Alertmanager notifies engineer

T+3 Logs reveal DB ping failure

T+5 Configuration corrected

T+6 Service recovers

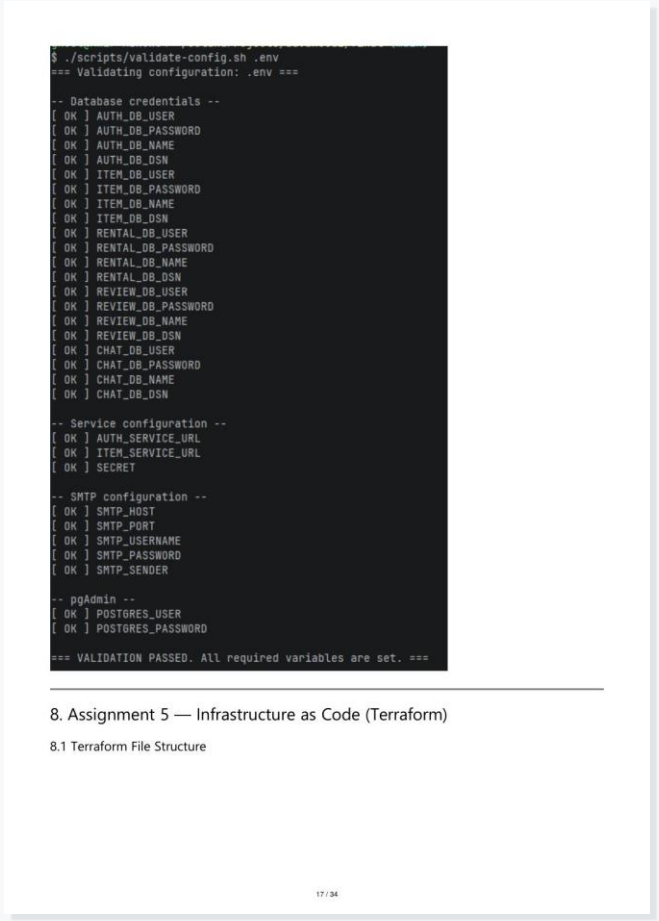
Root cause
Manual .env typo

Outcome
Recovered in 6 minutes

ASTANA IT UNIVERSITY

Remediation & prevention

The incident became automation



```
$ ./scripts/validate-config.sh .env
=== Validating configuration: .env ===

-- Database credentials --
[ OK ] AUTH_DB_USER
[ OK ] AUTH_DB_PASSWORD
[ OK ] AUTH_DB_NAME
[ OK ] AUTH_DB_DSN
[ OK ] ITEM_DB_USER
[ OK ] ITEM_DB_PASSWORD
[ OK ] ITEM_DB_NAME
[ OK ] ITEM_DB_DSN
[ OK ] RENTAL_DB_USER
[ OK ] RENTAL_DB_PASSWORD
[ OK ] RENTAL_DB_NAME
[ OK ] RENTAL_DB_DSN
[ OK ] REVIEW_DB_USER
[ OK ] REVIEW_DB_PASSWORD
[ OK ] REVIEW_DB_NAME
[ OK ] REVIEW_DB_DSN
[ OK ] CHAT_DB_USER
[ OK ] CHAT_DB_PASSWORD
[ OK ] CHAT_DB_NAME
[ OK ] CHAT_DB_DSN

-- Service configuration --
[ OK ] AUTH_SERVICE_URL
[ OK ] ITEM_SERVICE_URL
[ OK ] SECRET

-- SMTP configuration --
[ OK ] SMTP_HOST
[ OK ] SMTP_PORT
[ OK ] SMTP_USERNAME
[ OK ] SMTP_PASSWORD
[ OK ] SMTP_SENDER

-- pgAdmin --
[ OK ] POSTGRES_USER
[ OK ] POSTGRES_PASSWORD

=== VALIDATION PASSED. All required variables are set. ===
```

8. Assignment 5 — Infrastructure as Code (Terraform)

8.1 Terraform File Structure

17 / 34

Config validation
scripts/validate-config.sh checks all required variables and DSNs before deployment.

Health dependencies
Services wait for healthy databases before starting.

Per-service alerts
Down alerts detect outages within one minute.

Deployment discipline
CI/CD reduces manual production changes.

Load testing & capacity planning

Tests revealed bottlenecks before production scaling

Load script
Concurrent HTTP requests to all API endpoints.

Bottleneck
rental-service identified as the scaling priority.

Scaling path
Horizontal replicas through Swarm/Kubernetes HPA.

Capacity plan
Monitoring guides CPU, memory and replica decisions.

The goal was not just passing tests, but proving the system can be measured and scaled responsibly.

ASTANA IT UNIVERSITY

Repository deliverables

A complete production-style project, not just application code

`services/`	Go microservices	`frontend/`	React + TypeScript SPA
`gateway/`	Nginx reverse proxy	`observability/`	Prometheus + Grafana + alerts
`terraform/`	AWS infrastructure	`ansible/`	Server setup and deployment
`.github/workflows/`	CI/CD automation	`docker-compose.yaml`	Local full-stack orchestration

GitHub repository: silence99999/borrowly

Source code, workflows, infrastructure files and technical evidence are versioned together.

Final results

Requirements covered across the SRE lifecycle

Architecture
6 independent services + 6 isolated databases

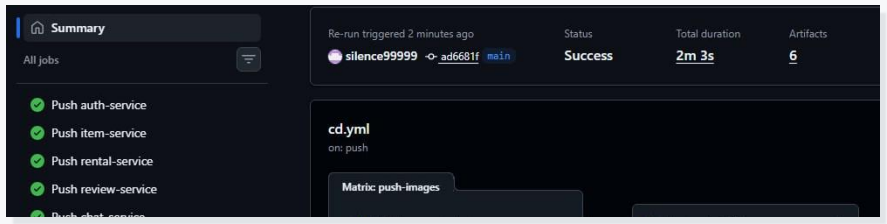
Automation
Terraform provisioning + Ansible setup

Delivery
19 checks pass on every push to main

Observability
Prometheus, Grafana, Alertmanager, 12 rules

Resilience
Incident detected <1 min and recovered in 6 min

Scaling
Load-tested and capacity-planned



ASTANA IT UNIVERSITY

Thank you

Borrowly proves the full path from code to reliable operation: build, deploy, monitor, respond, improve.

Reliable
health checks, SLOs, alerts

Repeatable
IaC + configuration management

Fast
2-minute CI/CD feedback loop

GitHub: github.com/silence99999/borrowly

ASTANA IT UNIVERSITY